International Cybersecurity and Digital Forensics Academy (ICDFA)

CIP-B102 Foundations of Computer and Digital Forensics

Lab 6: Data Carving with xxd, Binwalk and Scalpel

Forensic Lab Report

Item	Details
Student Name	Fuseini Imoru Kantuogaa
Registration Number	C11/26/DFIT/17291
Instructor's Name	Dr. Sarah James
Course Code	CIP-B102
Lab Number	Lab 6
Date Submitted	July 12, 2026
Working Environment	Kali Linux (kali@kali), working directory ~/CIP-B102_Lab6_Data_Carving

1. Objective

The purpose of Lab 6 is to develop practical digital forensic skills in data carving: recovering files and file fragments when normal file-system metadata is missing, damaged, or unreliable. This lab uses xxd to inspect binary content and file signatures at the byte level, The Sleuth Kit (TSK) to examine FAT file-system structures and recover files by inode, binwalk to detect and extract embedded file signatures inside container files, and scalpel to carve deleted files directly from a raw USB image using header/footer signatures. The exercise reinforces the difference between normal file recovery through metadata and true data carving from unallocated or unreferenced space, and it documents every command, output, hash value, and finding needed to reconstruct the analysis.

2. Tools and Environment

Tool	Version / Notes	Use in this Lab
OS / VM	Kali Linux (kali@kali)	Forensic working environment for all commands
xxd	Kali default build	Hex dump and reverse hex dump of the JPEG sample
The Sleuth Kit	img_stat, mmls, fsstat, fls, istat, icat	Direct inode/sector-level examination of FAT images
binwalk	Kali default build	Detection/extraction of embedded file signatures
scalpel	v1.60 (based on Foremost 0.69)	Header/footer based carving of deleted files from the USB image
7z / p7zip	7-Zip 26.01 (x64)	Extraction of the downloaded 120M.7z archive
md5sum / sha256sum	coreutils	Integrity verification before and after extraction

Working directory: ~/CIP-B102_Lab6_Data_Carving (created for this lab and used for every command below).

3. Evidence Sources

The following files were downloaded exactly as specified in the Data Carving slide material and used unmodified as evidence throughout the lab.

File	Source URL	Purpose
Ch01InChap01.dd	https://www.dropbox.com/s/nw23q14vzsykyup/Ch01InChap01.dd	FAT12 image for TSK inode/sector-level demonstration
J_ub_law.jpg	raw.githubusercontent.com/.../usb_file/J_ub_law.jpg	xxd header/tail signature analysis and hexdump reconstruction
120M.7z	raw.githubusercontent.com/.../usb_image/120M.7z	Deleted-file carving with scalpel after extraction

3.1 Hash Values of Downloaded Evidence

File	MD5	SHA-256
Ch01InChap01.dd	a117773bcf1fc88ec0ab8e0a349fbbcb	3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122
J_ub_law.jpg	83a360ac7f7e0ca318e5bfe39f95f137	238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468

120M.7z	dfe7b5424e54cd1bf50d5df47aceeb3c	2d2a3d93c9ec65bcad9f89c9894429ea72caa8add07726a3b96ec9a0ab6a58ce

(Hash values transcribed from the command output shown in Figure 3; refer to that screenshot as the authoritative source.)

4. Procedure

4.1 Preparation: Working Folder and Downloads

A dedicated working directory was created and the three evidence files were downloaded with wget, then verified by file type, size, and hash.

```
mkdir -p ~/CIP-B102_Lab6_Data_Carving cd ~/CIP-B102_Lab6_Data_Carving wget -q <Ch01InChap01.dd URL> -O Ch01InChap01.dd wget -q <J_ub_law.jpg URL> -O J_ub_law.jpg wget -q <120M.7z URL> -O 120M.7z
```

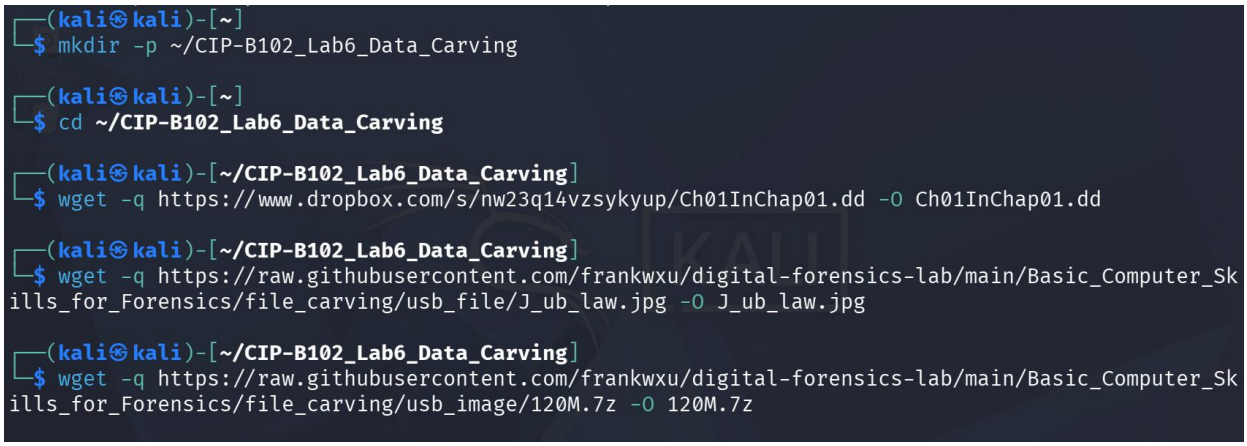

Figure 1. Working directory created and the three evidence files downloaded with wget.

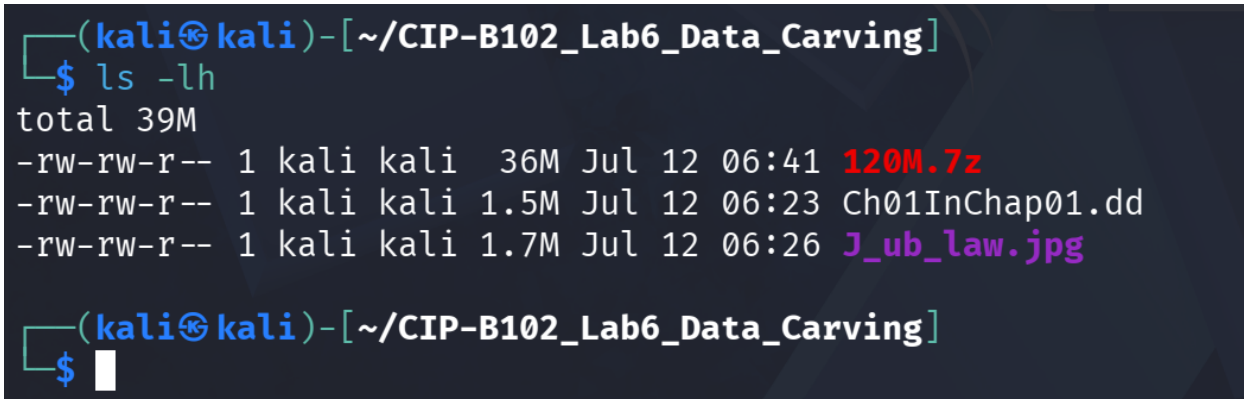

Figure 2. Directory listing (ls -lh) confirming all three files downloaded successfully.

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ file Ch01InChap01.dd J_ub_law.jpg 120M.7z
Ch01InChap01.dd: DOS/MBR boot sector, code offset 0×34+2, OEM-ID ",'k0nIHC" cached by Windows 9M, r
oot entries 224, sectors 2880 (volumes ≤32 MB), sectors/FAT 9, sectors/track 18, dos < 4.0 BootSec
tor (0), FAT (12 bit by descriptor+sectors), followed by FAT
J_ub_law.jpg:    JPEG image data, Exif standard: [TIFF image data, little-endian, direntries=16, wi
dth=4928, height=3280, bps=206, PhotometricInterpretation=RGB, manufacturer=NIKON CORPORATION, mode
l=NIKON D4, orientation=upper-left], baseline, precision 8, 1920×1080, components 3
120M.7z:         7-zip archive data, version 0.4

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ md5sum Ch01InChap01.dd J_ub_law.jpg 120M.7z
a117773bcf1fc88ec0ab8e0a349fbbcb  Ch01InChap01.dd
83a360ac7f7e0ca318e5bfe39f95f137  J_ub_law.jpg
dfe7b5424e54cd1bf50d5df47aceeb3c  120M.7z

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ sha256sum Ch01InChap01.dd J_ub_law.jpg 120M.7z
3ce8053e4f3d9c8ab98b3aadb2480685efb8e4980d34297b83bd5a09b1a7b122  Ch01InChap01.dd
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468  J_ub_law.jpg
2d2a3d93c9ec65bcad9f89c9894429ea72caa8add07726a3b96ec9a6ab6a58ce  120M.7z
```

Figure 3. file, md5sum, and sha256sum output identifying each file type and recording baseline hash values.

4.2 Part A: File Metadata vs. Data Carving (Conceptual)

Retrieving a file through file-system metadata means the file system's directory entries and allocation tables (e.g., the FAT) still point to valid, allocated data for that file, so the OS can locate and read it directly by name. Data carving instead ignores file-system metadata entirely and scans the raw bytes of a device or image for header signatures (e.g., 0xFFD8 for a JPEG's Start of Image, SOI) and footer/tail signatures (0xFFD9 for JPEG End of Image, EOI), extracting everything in between as a candidate file. This works even when a directory entry has been deleted or damaged, because the underlying data may still exist in sectors/blocks that the file system no longer marks as allocated (unallocated space) until it is overwritten.

Answers to the required questions:

- A deleted file can still be partially or fully recoverable because deleting a file typically removes or marks its directory entry and FAT chain as free, but the actual data in the associated clusters is not erased until those clusters are reused and overwritten by new data.
- A file name can be lost even when content is recovered because the name lives in file-system metadata (the directory entry), while the content lives in data blocks/clusters. Carving tools that scan for header/footer signatures recover the byte content but have no metadata to read the original name from, so carved files are typically given generic sequential names.

- Normal file recovery reads a file through intact, valid file-system metadata (directory entry + allocation table); data carving reconstructs files purely from recognizable byte patterns in raw data, independent of whether any metadata for that file still exists.
- The JPEG markers used to bound a file are SOI (Start of Image) 0xFFD8 at the beginning and EOI (End of Image) 0xFFD9 at the end.

4.3 Part B: Examining the JPG with xxd

```
xxd J_ub_law.jpg | head xxd J_ub_law.jpg | tail xxd -l 120 J_ub_law.jpg xxd -p J_ub_law.jpg > J_ub_law_hexdump.txt grep -o "ffd8" J_ub_law_hexdump.txt | head grep -o "ffd9" J_ub_law_hexdump.txt | tail xxd -p J_ub_law.jpg | tr -d "\n" > J_ub_law_hexdump_singleline.txt grep -o "ffd8" J_ub_law_hexdump_singleline.txt grep -o "ffd9" J_ub_law_hexdump_singleline.txt
```

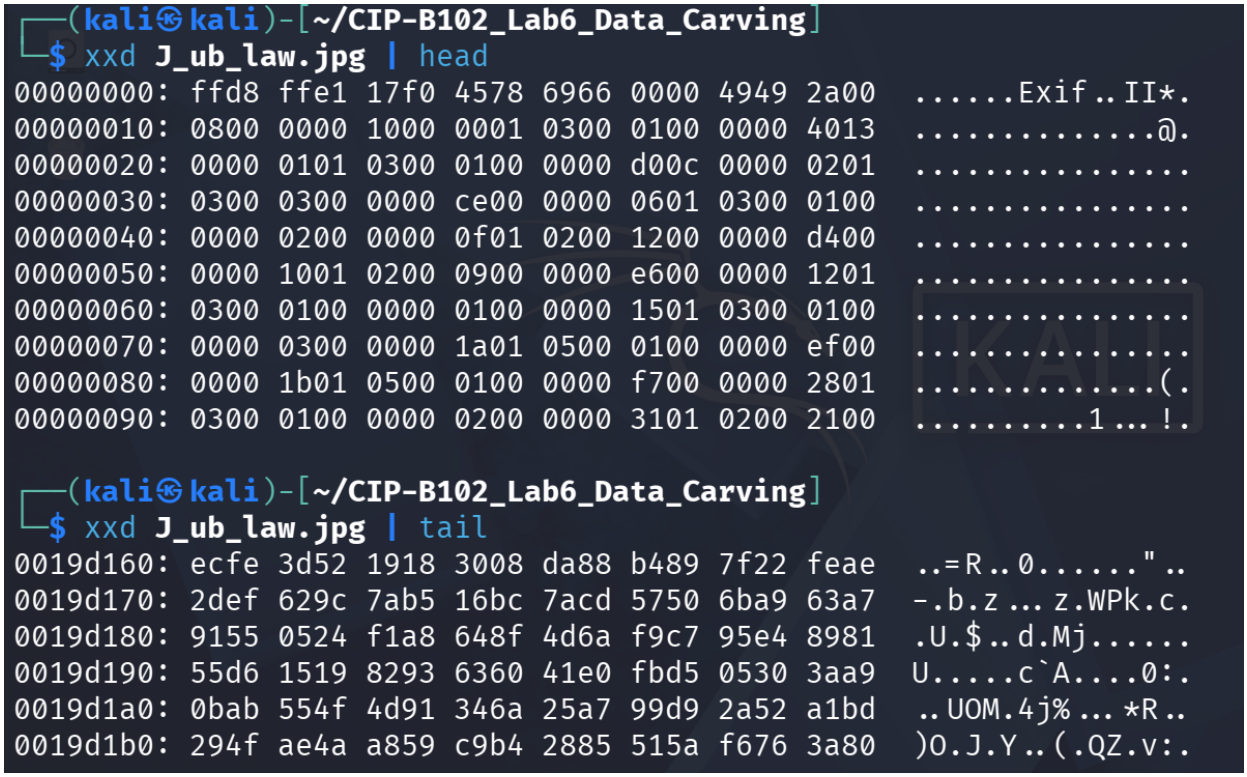

Figure 4. xxd J_ub_law.jpg | head and | tail, showing the ffd8 SOI marker at the very start of the file and unstructured compressed data at the end.

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ xxd -l 120 J_ub_law.jpg
00000000: ffd8 ffe1 17f0 4578 6966 0000 4949 2a00  ......Exif..II*.
00000010: 0800 0000 1000 0001 0300 0100 0000 4013  ..............@.
00000020: 0000 0101 0300 0100 0000 d00c 0000 0201  ................
00000030: 0300 0300 0000 ce00 0000 0601 0300 0100  ................
00000040: 0000 0200 0000 0f01 0200 1200 0000 d400  ................
00000050: 0000 1001 0200 0900 0000 e600 0000 1201  ................
00000060: 0300 0100 0000 0100 0000 1501 0300 0100  ................
00000070: 0000 0300 0000 1a01                      ........

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ xxd -p J_ub_law.jpg > J_ub_law_hexdump.txt

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ grep -o "ffd8" J_ub_law_hexdump.txt | head
ffd8
ffd8
ffd8
ffd8
ffd8
ffd8
```

Figure 5. xxd -l 120 showing the first 120 bytes and the beginning of the plain hexdump generation (xxd -p).

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ grep -o "ffd9" J_ub_law_hexdump.txt | tail
ffd9
ffd9
ffd9
ffd9
ffd9
ffd9
ffd9
ffd9
ffd9
ffd9

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ xxd -p J_ub_law.jpg | tr -d "\n" > J_ub_law_hexdump_singleline.txt

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ grep -o "ffd8" J_ub_law_hexdump_singleline.txt
ffd8
ffd8
ffd8
ffd8
```

Figure 6. grep search for the ffd9 tail signature in the plain hexdump, and creation of a single-line (no newline) hexdump so signatures are not split across line breaks.

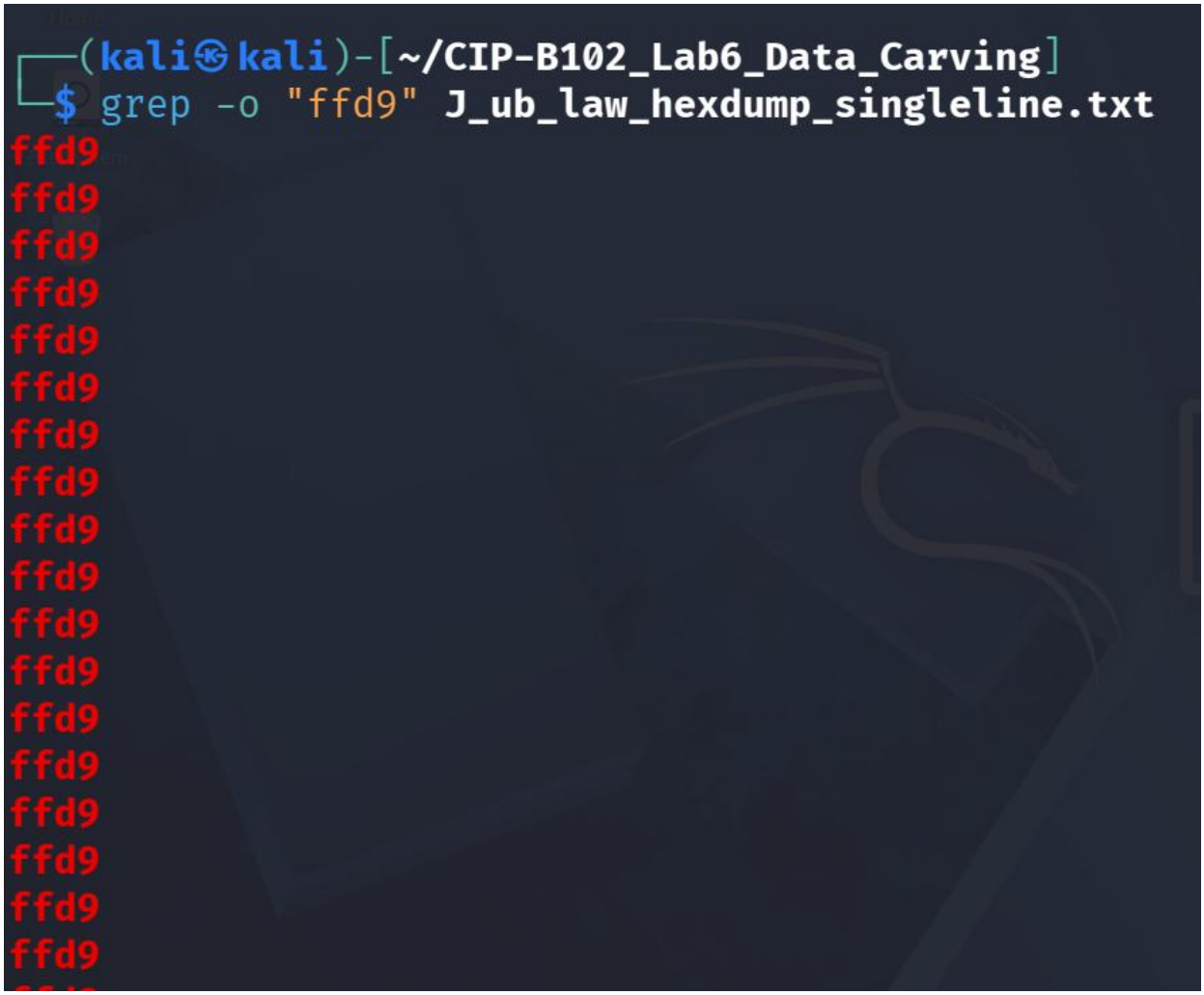

Figure 7. Continued search for the ffd9 footer signature across the single-line hexdump.

The single JPEG in this sample only contains one true SOI (0xFFD8) at offset 0 and one true EOI (0xFFD9) at the very end of the file; the additional ffd8/ffd9 matches returned by grep on the raw (non single-line) hexdump are coincidental byte patterns inside the compressed image data and metadata, which is why the single-line search is the more reliable check for the true header/footer boundary.

The plain hexdump was then converted back into a binary JPEG using xxd -r -p, and the reconstructed file was compared against the original by hash to confirm the round-trip was lossless.

```
xxd -r -p J_ub_law_hexdump.txt J_ub_law_reverse_dump.jpg file J_ub_law_reverse_dump.jpg md5sum J_ub_law.jpg J_ub_law_reverse_dump.jpg sha256sum J_ub_law.jpg J_ub_law_reverse_dump.jpg
```

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ xxd -r -p J_ub_law_hexdump.txt J_ub_law_reverse_dump.jpg

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ file J_ub_law_reverse_dump.jpg
J_ub_law_reverse_dump.jpg: JPEG image data, Exif standard: [TIFF image data, little-endian, direntr
ies=16, width=4928, height=3280, bps=206, PhotometricInterpretation=RGB, manufacturer=NIKON CORPORA
TION, model=NIKON D4, orientation=upper-left], baseline, precision 8, 1920×1080, components 3

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ md5sum J_ub_law.jpg J_ub_law_reverse_dump.jpg
83a360ac7f7e0ca318e5bfe39f95f137  J_ub_law.jpg
83a360ac7f7e0ca318e5bfe39f95f137  J_ub_law_reverse_dump.jpg

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ sha256sum J_ub_law.jpg J_ub_law_reverse_dump.jpg
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468  J_ub_law.jpg
238ff34393c50e52c0e8b14fcff8ec7dc29e23914dbc435f8ef998d172a91468  J_ub_law_reverse_dump.jpg
```

Figure 8. Reverse hexdump reconstruction (xxd -r -p) and hash comparison: J_ub_law.jpg and J_ub_law_reverse_dump.jpg produced identical MD5 and SHA-256 values, confirming an exact reconstruction.

Expected observation confirmed: the MD5 (83a360ac7f7e0ca318e5bfe39f95f137) and SHA-256 hash of the original and the reverse-dumped file matched exactly.

4.4 Part C: Metadata-Like Strings Inside the JPG

```
xxd -s 0x100 -l 0x9f J_ub_law.jpg strings J_ub_law.jpg | head -50 strings J_ub_law.jpg | grep -Ei "nikon|photoshop|adobe|date|copyright|author|name"
```

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ xxd -s 0×100 -l 0×9f J_ub_law.jpg
00000100: 2700 00c0 c62d 0010 2700 0041 646f 6265  '....-..'..Adobe
00000110: 2050 686f 746f 7368 6f70 2032 312e 3120   Photoshop 21.1
00000120: 284d 6163 696e 746f 7368 2900 3230 3230  (Macintosh).2020
00000130: 3a30 383a 3230 2031 303a 3230 3a30 3500  :08:20 10:20:05.
00000140: 484f 5741 5244 204b 4f52 4e00 3200 3000  HOWARD KORN.2.0.
00000150: 3100 3300 2c00 2000 6300 6100 6d00 7000  1.3.,. .c.a.m.p.
00000160: 7500 7300 2c00 2000 4d00 6500 7200 7200  u.s.,. .M.e.r.r.
00000170: 6900 6300 6b00 2000 5300 6300 6800 6f00  i.c.k. .S.c.h.o.
00000180: 6f00 6c00 2000 6f00 6600 2000 4200 7500  o.l. .o.f. .B.u.
00000190: 7300 6900 6e00 6500 7300 7300 0000 00    s.i.n.e.s.s....
```

Figure 9. xxd -s 0x100 -l 0x9f showing the metadata region of the JPEG, including readable Adobe/Photoshop and author-related text.

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ strings J_ub_law.jpg | head -50
Exif
NIKON CORPORATION
NIKON D4
Adobe Photoshop 21.1 (Macintosh)
2020:08:20 10:20:05
HOWARD KORN
0221
2013:10:08 18:56:21
2013:10:08 18:56:21
2027750
17.0-35.0 mm f/2.8
Adobe_CM
Adobe
b34r
7GWgw
AQaq"
dEU6te
'7GWgw
```

Figure 10. strings output (first 50 lines) revealing camera make/model, editing software, and embedded dates.

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ strings J_ub_law.jpg | grep -Ei "nikon|photoshop|adobe|date|copyright|author|name"
NIKON CORPORATION
NIKON D4
Adobe Photoshop 21.1 (Macintosh)
Adobe_CM
Adobe
"Photoshop 3.0
printerNameTEXT
Adobe_CM
Adobe
http://ns.adobe.com/xap/1.0/
" id="W5M0MpCehiHzreSzNTczkc9d"?> <x:xmpmeta xmlns:x="adobe:ns:meta/" x:xmptk="Adobe XMP Core 6.0-c
002 79.164352, 2020/01/30-15:50:38        "> <rdf:RDF xmlns:rdf="http://www.w3.org/1999/02/22-rdf-s
yntax-ns#"> <rdf:Description rdf:about="" xmlns:aux="http://ns.adobe.com/exif/1.0/aux/" xmlns:dc="h
ttp://purl.org/dc/elements/1.1/" xmlns:photoshop="http://ns.adobe.com/photoshop/1.0/" xmlns:xmp="ht
tp://ns.adobe.com/xap/1.0/" xmlns:xmpMM="http://ns.adobe.com/xap/1.0/mm/" xmlns:stRef="http://ns.ad
obe.com/xap/1.0/sType/ResourceRef#" xmlns:stEvt="http://ns.adobe.com/xap/1.0/sType/ResourceEvent#"
xmlns:crs="http://ns.adobe.com/camera-raw-settings/1.0/" xmlns:lr="http://ns.adobe.com/lightroom/1.
0/" aux:ApproximateFocusDistance="4294967295/1" aux:ImageNumber="115422" aux:Lens="17.0-35.0 mm f/2
.8" aux:LensID="99" aux:LensInfo="170/10 350/10 28/10 28/10" aux:SerialNumber="2027750" dc:format="
```

Figure 11. Filtered strings search (grep -Ei) isolating camera, software, and copyright-related identifiers.

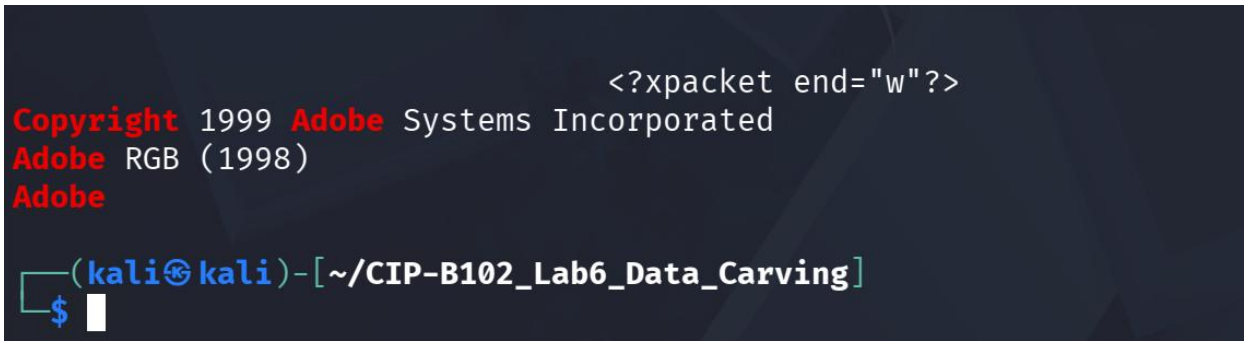

Figure 12. Continued filtered strings output, including the embedded 'Copyright 1999 Adobe Systems Incorporated' string and Adobe RGB colour-profile reference.

Strings/metadata found in J_ub_law.jpg:

Field	Value Found	Notes
Camera manufacturer	NIKON CORPORATION	From embedded Exif/TIFF tags
Camera model	NIKON D4	From embedded Exif/TIFF tags
Editing software	Adobe Photoshop 21.1 (Macintosh)	From Photoshop/XMP metadata block
Software-recorded date	2020:08:20 10:20:05	Photoshop edit timestamp
Name string	HOWARD KORN	Plain-text name found via strings
Lens	17.0-35.0 mm f/2.8	From XMP aux:Lens tag
Copyright	Copyright 1999 Adobe Systems Incorporated	Embedded copyright string, also flagged by binwalk
Original capture date	2013:10:08 18:56:21	EXIF DateTimeOriginal-style field

4.5 Part D: Direct Data-Unit Access in the DD Image with TSK

img_stat Ch01InChap01.dd mmls Ch01InChap01.dd fsstat Ch01InChap01.dd | head -40 fls Ch01InChap01.dd fls -d Ch01InChap01.dd

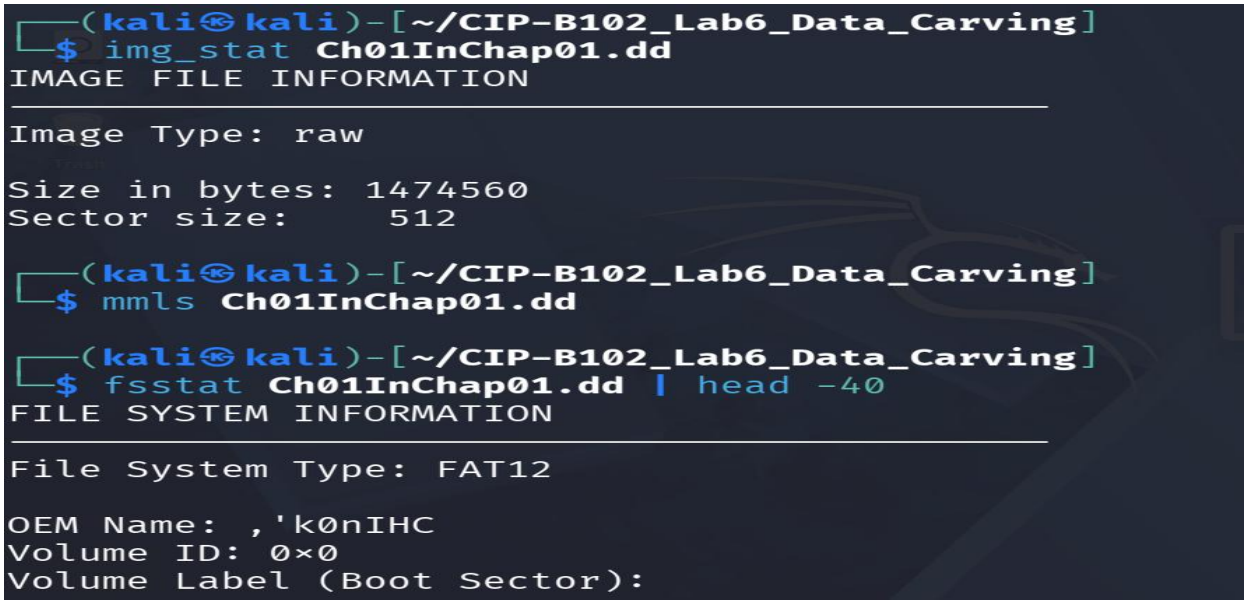

Figure 13. img_stat and the start of fsstat output: raw image, 1,474,560 bytes, 512-byte sectors, FAT12 file system.

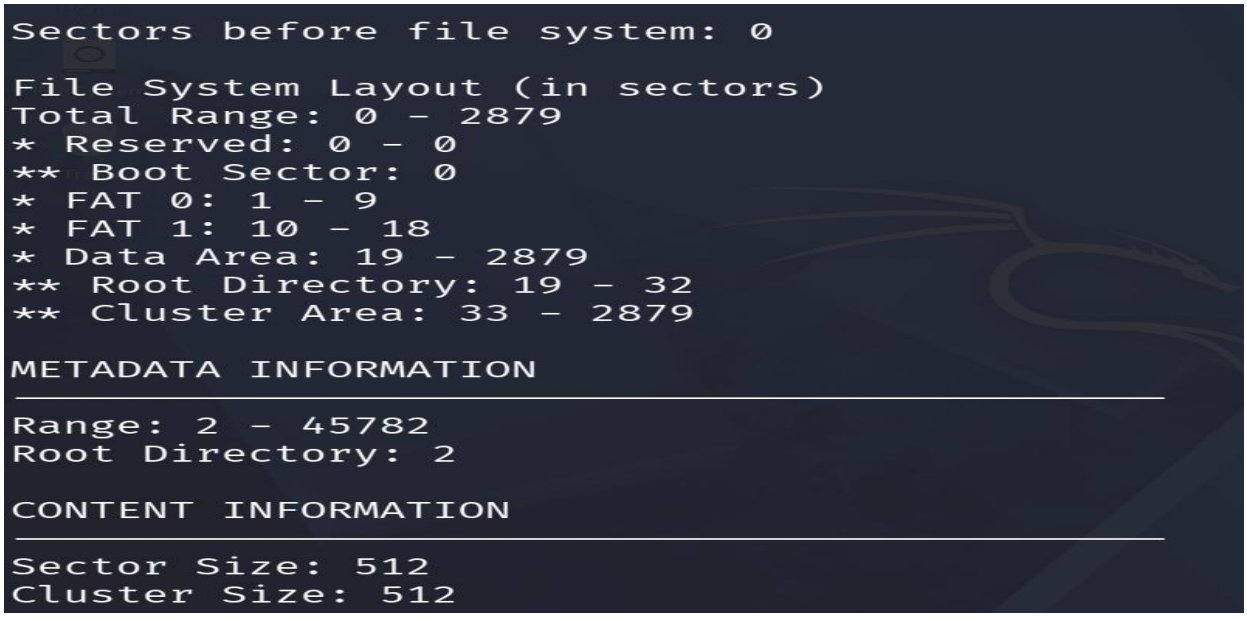

Figure 14. fsstat file-system layout: reserved area, two FAT copies (sectors 1-9 and 10-18), root directory and data/cluster area (sectors 19-2879).

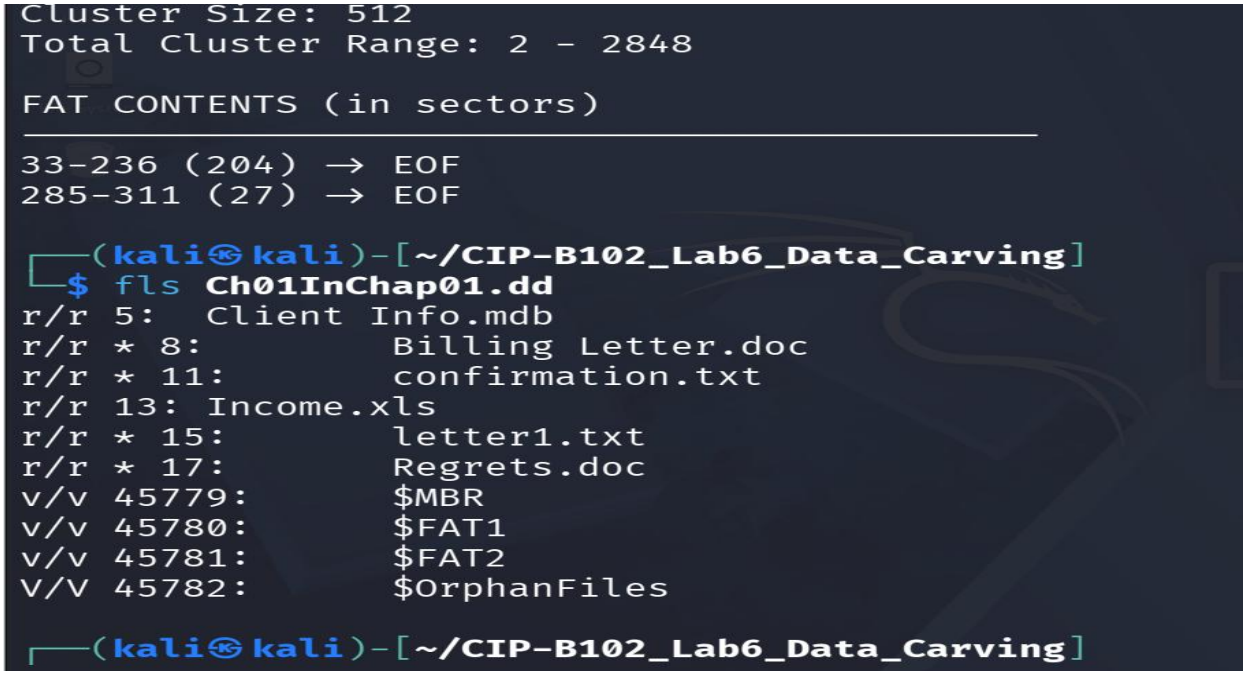

*Figure 15. fsstat FAT chain summary and fls listing of the root directory, showing allocated and deleted (marked *) entries.*

fls confirmed six file entries in the root directory, four of which are marked with * (unallocated / deleted): Billing Letter.doc (8), confirmation.txt (11), letter1.txt (15), and Regrets.doc (17). Client Info.mdb (5) and Income.xls (13) remained allocated.

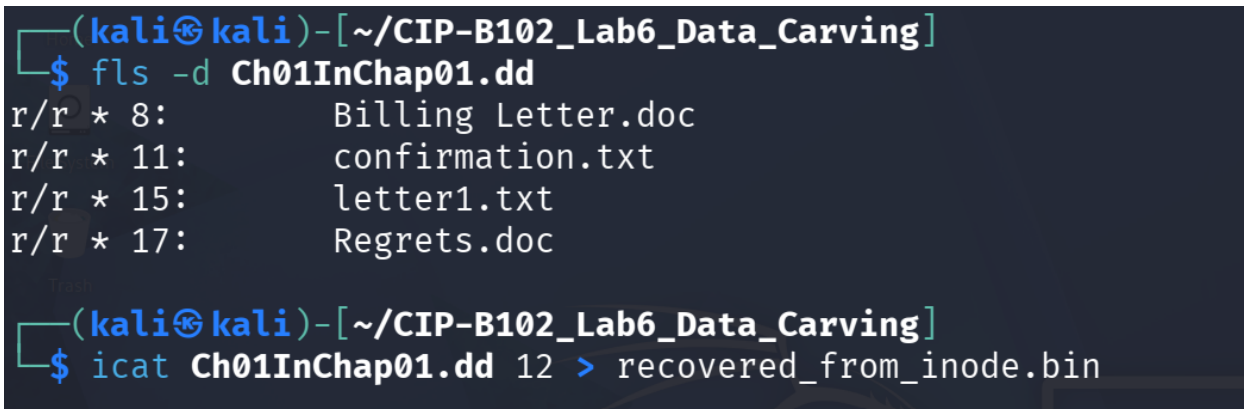

Figure 16. fls -d isolating only the deleted entries, followed by the start of icat recovery using the inode/entry number.

icat was used to recover file content directly from the inode/entry numbers identified above, and the recovered files were verified with file:

icat Ch01InChap01.dd 8 > Billing_Letter.doc file Billing_Letter.doc icat Ch01InChap01.dd 11 > confirmation.txt file confirmation.txt icat Ch01InChap01.dd 15 > letter1.txt file letter1.txt

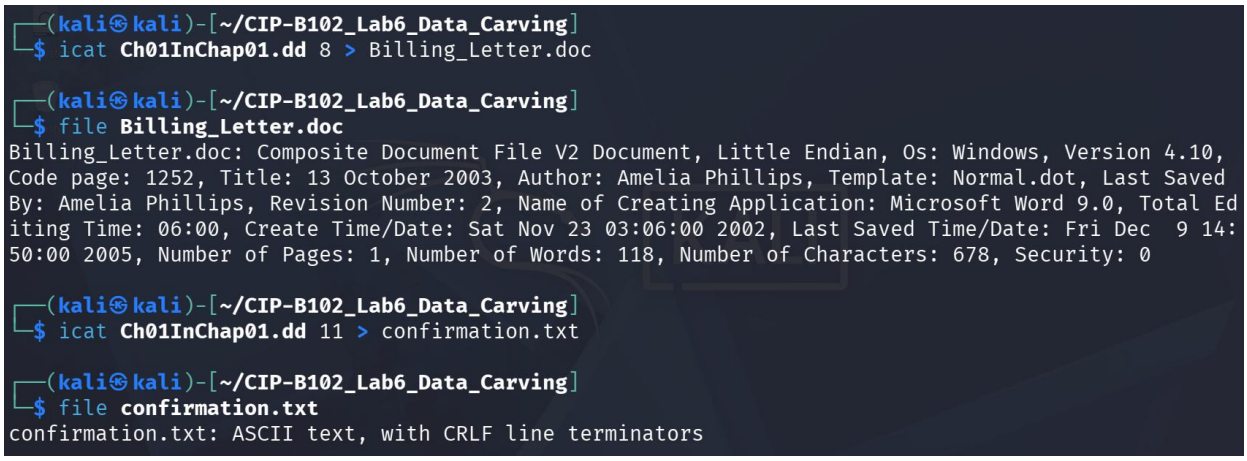

Figure 17. icat recovery of entry 8 (Billing_Letter.doc), confirmed by file as a genuine MS Word 9.0 document with embedded author metadata (Amelia Phillips); icat recovery of entry 11 (confirmation.txt).

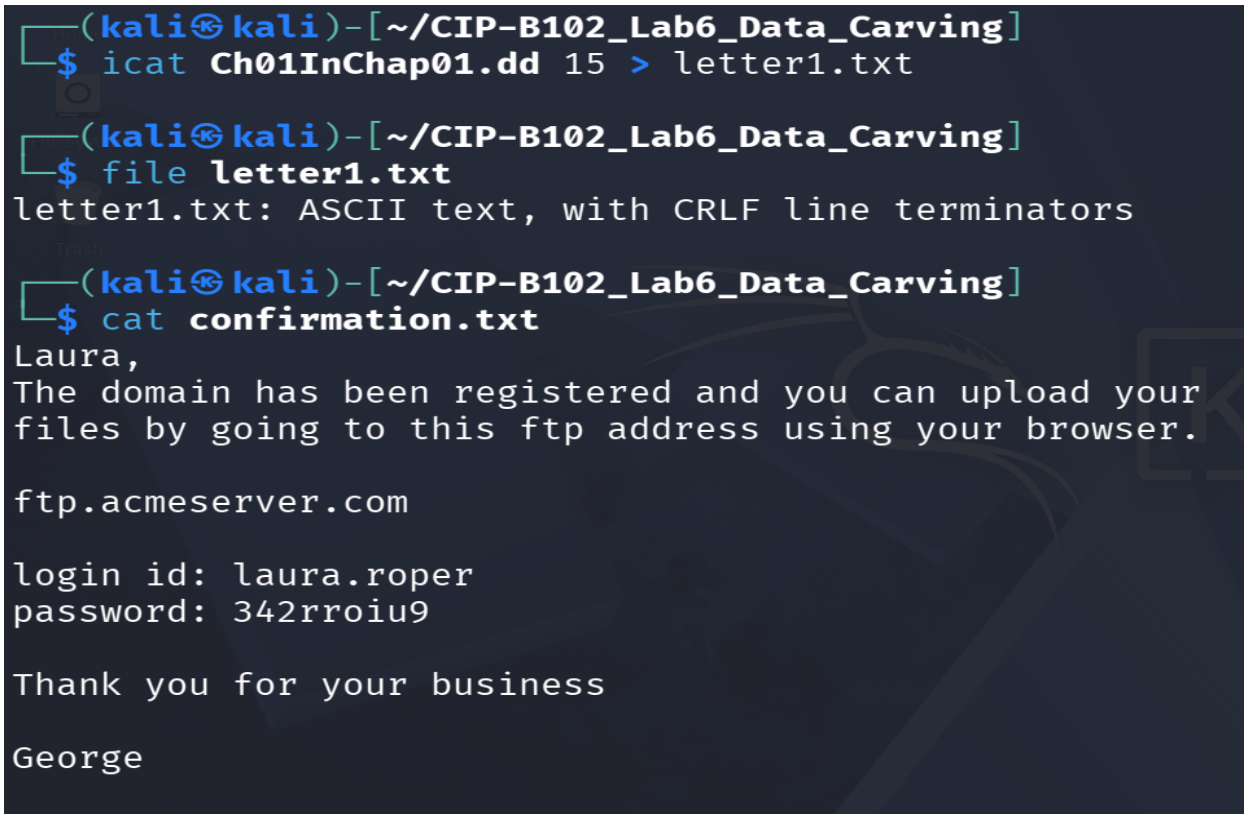

Figure 18. icat recovery of entry 15 (letter1.txt) and the recovered content of confirmation.txt, an FTP login handoff message.

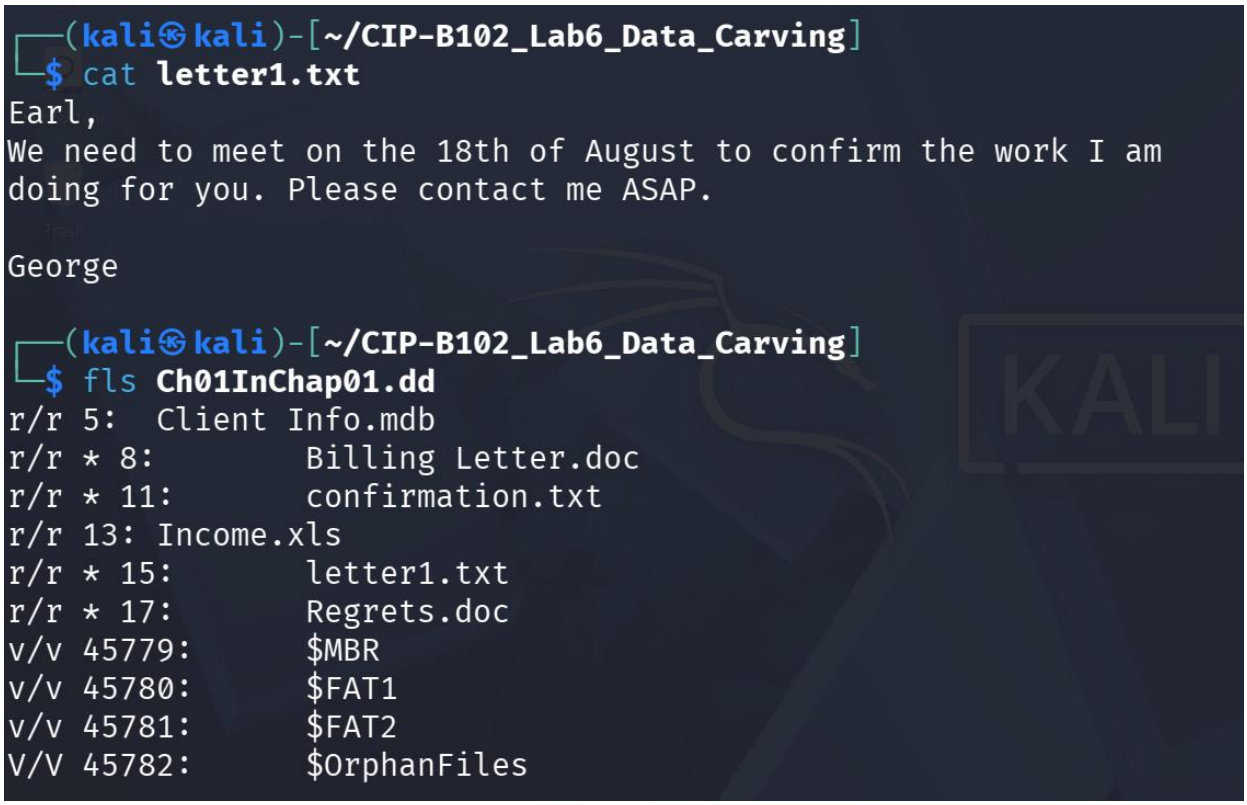

Figure 19. Recovered content of letter1.txt (a short meeting-reminder message) and a repeat of the fls listing for reference.

Recovered file contents:

Recovered File	Content Summary
confirmation.txt	Message from 'George' to 'Laura' providing an FTP address, login ID (laura.roper) and password (342rroiu9) for uploading files.
letter1.txt	Short note from 'George' to 'Earl' about meeting on the 18th of August to confirm work in progress.

istat was then used against each deleted entry to record its metadata (size, name, timestamps, and sector allocation) directly from the directory entry, independent of whether the entry is still marked allocated:

```
istat Ch01InChap01.dd 8 istat Ch01InChap01.dd 11 istat Ch01InChap01.dd 15 istat Ch01InChap01.dd 17
```

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ istat Ch01InChap01.dd 8
Directory Entry: 8
Not Allocated
File Attributes: File, Archive
Size: 24064
Name: _ILLIN~1.DOC

Directory Entry Times:
Written:        2005-12-09 06:50:28 (EST)
Accessed:       2005-12-09 00:00:00 (EST)
Created:        2005-12-09 06:59:05 (EST)

Sectors:
237 238 239 240 241 242 243 244
245 246 247 248 249 250 251 252
253 254 255 256 257 258 259 260
261 262 263 264 265 266 267 268
269 270 271 272 273 274 275 276
277 278 279 280 281 282 283
```

Figure 20. istat for entry 8: _ILLIN~1.DOC (Billing Letter.doc), 24,064 bytes, not allocated, with full written/accessed/created timestamps and its sector run (237-283).

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ istat Ch01InChap01.dd 11
Directory Entry: 11
Not Allocated
File Attributes: File, Archive
Size: 227
Name: _ONFIR~1.TXT

Directory Entry Times:
Written:        2005-12-09 06:52:58 (EST)
Accessed:       2005-12-09 00:00:00 (EST)
Created:        2005-12-09 06:59:06 (EST)

Sectors:
284
```

Figure 21. istat for entry 11: _ONFIR~1.TXT (confirmation.txt), 227 bytes, not allocated, single sector (284).

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ istat Ch01InChap01.dd 15
Directory Entry: 15
Not Allocated
File Attributes: File, Archive
Size: 121
Name: _ETTER1.TXT

Directory Entry Times:
Written:        2005-12-09 06:51:50 (EST)
Accessed:       2005-12-09 00:00:00 (EST)
Created:        2005-12-09 06:59:09 (EST)

Sectors:
312
```

Figure 22. istat for entry 15: _ETTER1.TXT (letter1.txt), 121 bytes, not allocated, single sector (312).

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ istat Ch01InChap01.dd 17
Directory Entry: 17
Not Allocated
File Attributes: File, Archive
Size: 23552
Name: _EGRETS.DOC

Directory Entry Times:
Written:        2005-12-09 06:50:52 (EST)
Accessed:       2005-12-09 00:00:00 (EST)
Created:        2005-12-09 06:59:09 (EST)

Sectors:
313 314 315 316 317 318 319 320
321 322 323 324 325 326 327 328
329 330 331 332 333 334 335 336
337 338 339 340 341 342 343 344
345 346 347 348 349 350 351 352
353 354 355 356 357 358
```

Figure 23. istat for entry 17: _EGRETS.DOC (Regrets.doc), 23,552 bytes, not allocated, sector run (313-358).

Note: TSK still displays a name for each of these entries (e.g., _ILLIN~1.DOC) because the short 8.3 directory entry itself was still present and only flagged as 'not allocated' this is a metadata-assisted recovery, not true header/footer carving, which is why names and timestamps were still available here but were not available later for the scalpel-carved JPEGs.

4.6 Part E: Embedded File Discovery with Binwalk

The slide's manual dd-based extraction example depends on an instructor-provided File_carving.docx, which was not included among the slide's download URLs and was not available in this environment

(confirmed by ls: cannot access 'File_carving.docx'). Per the lab instructions, the binwalk practice was therefore completed using J_ub_law.jpg and 120M.7z instead.

```
ls -lh File_carving.docx  # not available binwalk J_ub_law.jpg file J_ub_law.jpg
```

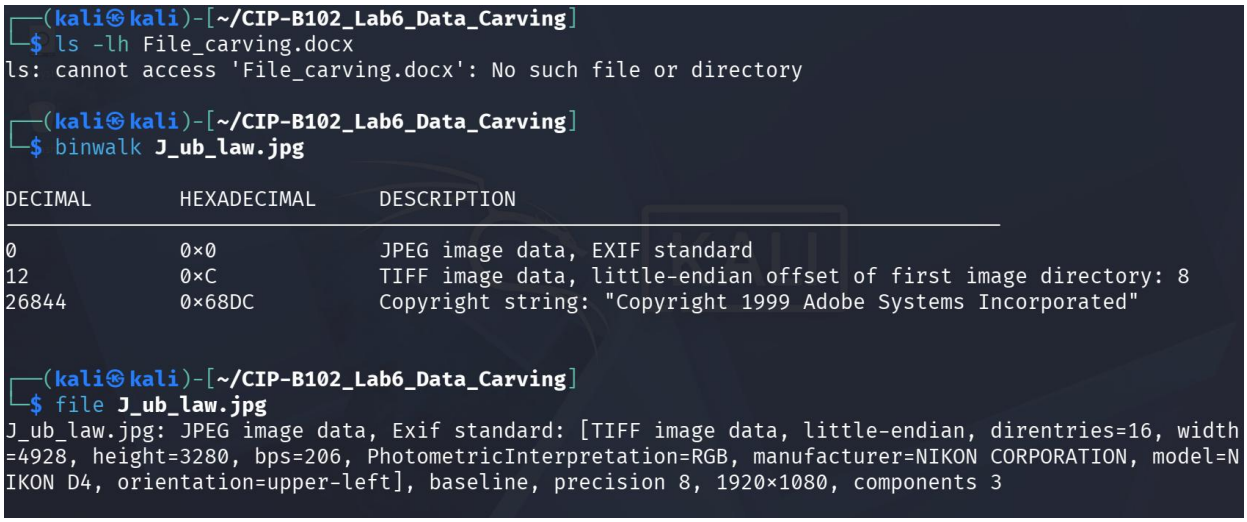

Figure 24. File_carving.docx confirmed unavailable; binwalk on J_ub_law.jpg identifying the EXIF/TIFF header at offset 0 and an embedded copyright string ('Copyright 1999 Adobe Systems Incorporated') at offset 0x68DC.

```
binwalk Ch01InChap01.dd binwalk -e Ch01InChap01.dd ls -lah _Ch01InChap01.dd.extracted
```

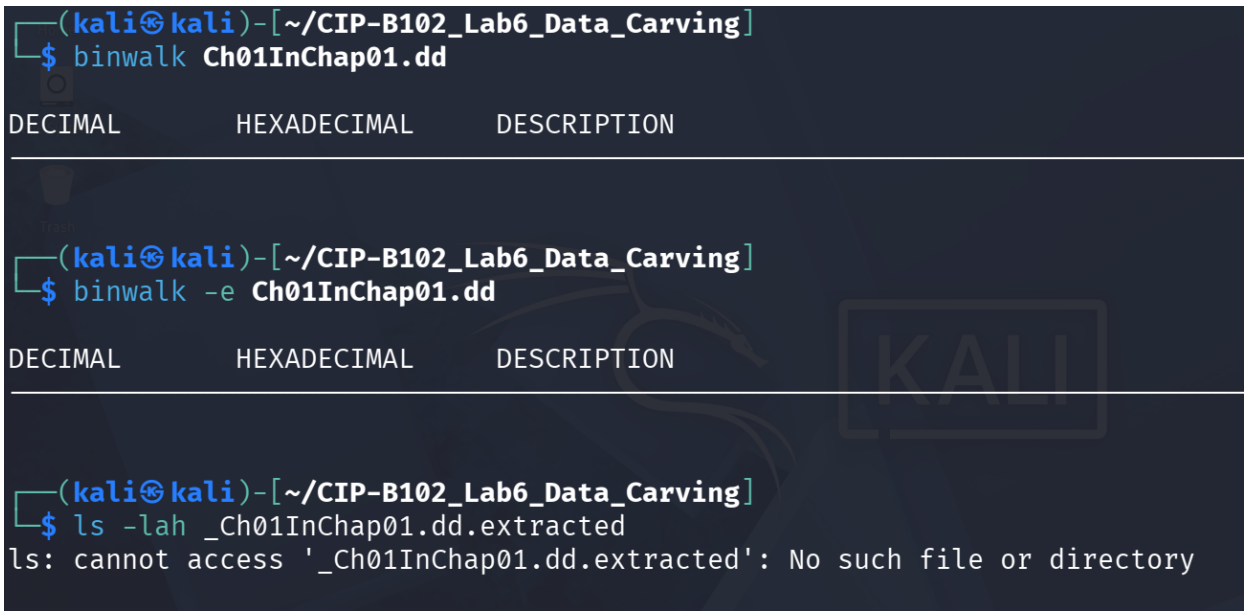

Figure 25. binwalk against Ch01InChap01.dd returned no signature matches, and binwalk -e produced no extraction folder, consistent with a plain FAT12 boot/data image containing only office documents and text files rather than any recognizable embedded container format.

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ file Ch01InChap01.dd
Ch01InChap01.dd: DOS/MBR boot sector, code offset 0x34+2, OEM-ID ",'k0nIHC" cached by Windows 9M, r
oot entries 224, sectors 2880 (volumes ⩽32 MB), sectors/FAT 9, sectors/track 18, dos < 4.0 BootSec
tor (0), FAT (12 bit by descriptor+sectors), followed by FAT

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ img_stat Ch01InChap01.dd
IMAGE FILE INFORMATION
--------------------------------------------
Image Type: raw

Size in bytes: 1474560
Sector size:    512

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ mmls Ch01InChap01.dd
```

Figure 26. Re-confirmation of Ch01InChap01.dd's file type, image size, and partition table for cross-reference with the binwalk result.

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ fls -r Ch01InChap01.dd
r/r 5:  Client Info.mdb
r/r * 8:        Billing Letter.doc
r/r * 11:       confirmation.txt
r/r 13: Income.xls
r/r * 15:       letter1.txt
r/r * 17:       Regrets.doc
v/v 45779:      $MBR
v/v 45780:      $FAT1
v/v 45781:      $FAT2
V/V 45782:      $OrphanFiles
```

Figure 27. fls -r (recursive) listing of Ch01InChap01.dd for completeness, showing the same six entries and system metadata files ($MBR, $FAT1, $FAT2, $OrphanFiles).

binwalk 120M.7z file 120M.7z binwalk -e 120M.7z ls -lah _120M.7z.extracted

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ binwalk 120M.7z

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0x0             7-zip archive data, version 0.4
1396360       0x154E88        Zip archive data, at least v1.0 to extract, compressed size: 190030,
uncompressed size: 190030, name: ppt/fonts/font4.fntdata
1586455       0x183517        Zip archive data, at least v1.0 to extract, compressed size: 18171, u
ncompressed size: 18171, name: ppt/fonts/font5.fntdata
1604679       0x187C47        Zip archive data, at least v1.0 to extract, compressed size: 34163, u
ncompressed size: 34163, name: ppt/fonts/font6.fntdata
1638898       0x1901F2        Zip archive data, at least v1.0 to extract, compressed size: 172668,
uncompressed size: 172668, name: ppt/fonts/font7.fntdata
2445986       0x2552A2        Zip archive data, at least v1.0 to extract, compressed size: 88758, u
ncompressed size: 88758, name: ppt/media/image14.png
3666842       0x37F39A        JBOOT STAG header, image id: 0, timestamp 0x1EFB070E, image size: 265
5696701 bytes, image JBOOT checksum: 0xFFDC, header JBOOT checksum: 0x9300
23522902      0x166EE56       JBOOT SCH2 kernel header, compression type: none, Entry Point: 0xEEE8
895A, image size: 1608501141 bytes, data CRC: 0x6977FA7D, Data Address: 0x24241959, rootfs offset:
0x88BA0829, rootfs size: 1182421830 bytes, rootfs CRC: 0xD70FC9DD, header CRC: 0x3ECF7B1F, header s
```

Figure 28. binwalk on 120M.7z: the true 7-zip archive header at offset 0, followed by several nested Zip archive entries (ppt/fonts/.fntdata, ppt/media/image14.png) and JBOOT-pattern matches.*

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ ls -lh 120M.7z
-rw-rw-r-- 1 kali kali 36M Jul 12 06:41 120M.7z

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ binwalk 120M.7z

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------
0             0×0             7-zip archive data, version 0.4
1396360       0×154E88        Zip archive data, at least v1.0 to extract, compressed size: 190030,
uncompressed size: 190030, name: ppt/fonts/font4.fntdata
1586455       0×183517        Zip archive data, at least v1.0 to extract, compressed size: 18171, u
ncompressed size: 18171, name: ppt/fonts/font5.fntdata
1604679       0×187C47        Zip archive data, at least v1.0 to extract, compressed size: 34163, u
ncompressed size: 34163, name: ppt/fonts/font6.fntdata
1638898       0×1901F2        Zip archive data, at least v1.0 to extract, compressed size: 172668,
uncompressed size: 172668, name: ppt/fonts/font7.fntdata
2445986       0×2552A2        Zip archive data, at least v1.0 to extract, compressed size: 88758, u
ncompressed size: 88758, name: ppt/media/image14.png
3666842       0×37F39A        JBOOT STAG header, image id: 0, timestamp 0×1EFB070E, image size: 265
```

Figure 29. Continued binwalk output on 120M.7z listing further nested Zip archive/font-data signatures inside the compressed stream.

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ file 120M.7z
120M.7z: 7-zip archive data, version 0.4

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ binwalk -e 120M.7z

DECIMAL       HEXADECIMAL     DESCRIPTION
--------------------------------------------------------------------------------

WARNING: Extractor.execute failed to run external extractor 'jar xvf '%e'': [Errno 2] No such file
or directory: 'jar', 'jar xvf '%e'' might not be installed correctly
1396360       0×154E88        Zip archive data, at least v1.0 to extract, compressed size: 190030,
uncompressed size: 190030, name: ppt/fonts/font4.fntdata
1586455       0×183517        Zip archive data, at least v1.0 to extract, compressed size: 18171, u
ncompressed size: 18171, name: ppt/fonts/font5.fntdata
1604679       0×187C47        Zip archive data, at least v1.0 to extract, compressed size: 34163, u
ncompressed size: 34163, name: ppt/fonts/font6.fntdata
1638898       0×1901F2        Zip archive data, at least v1.0 to extract, compressed size: 172668,
uncompressed size: 172668, name: ppt/fonts/font7.fntdata
2445986       0×2552A2        Zip archive data, at least v1.0 to extract, compressed size: 88758, u
```

Figure 30. file confirming 120M.7z as a 7-zip archive, and binwalk -e attempting automatic extraction (the 'jar' extractor was not installed, so the JBOOT/font matches were not unpacked automatically).

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ ls -lah _120M.7z.extracted
total 44M
drwxrwxr-x 2 kali kali 4.0K Jul 12 07:33 .
drwxrwxr-x 3 kali kali 4.0K Jul 12 07:33 ..
-rw-rw-r-- 1 kali kali  34M Jul 12 07:33 154E88.zip
-rw-rw-r-- 1 kali kali  12K Jul 12 07:33 2085526
-rw-rw-r-- 1 kali kali 2.5M Jul 12 07:33 2085526.zlib
-rw-rw-r-- 1 kali kali  11K Jul 12 07:33 208825A
-rw-rw-r-- 1 kali kali 2.5M Jul 12 07:33 208825A.zlib
-rw-rw-r-- 1 kali kali  11K Jul 12 07:33 208A93F
-rw-rw-r-- 1 kali kali 2.5M Jul 12 07:33 208A93F.zlib
-rw-rw-r-- 1 kali kali  17K Jul 12 07:33 208B5C8
-rw-rw-r-- 1 kali kali 2.5M Jul 12 07:33 208B5C8.zlib

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$
```

Figure 31. Contents of the partial _120M.7z.extracted folder: only a handful of small zip/zlib fragments were produced, not the full usb_fat_carving.001 image.

The PPT-font and JBOOT-style signatures reported by binwalk inside 120M.7z are false positives generated by scanning the raw compressed LZMA2 byte stream for coincidental byte patterns; they do not correspond to real embedded PowerPoint or firmware files. This was confirmed in the next step, where 7z l shows the archive's true contents are only usb_fat_carving.001 and a small companion .txt file.

4.7 Part F: Preparing and Examining the USB Image

7z l 120M.7z 7z e 120M.7z ls -lh file * | tee file_types_lab6.txt md5sum * | tee md5_after_extract_lab6.txt

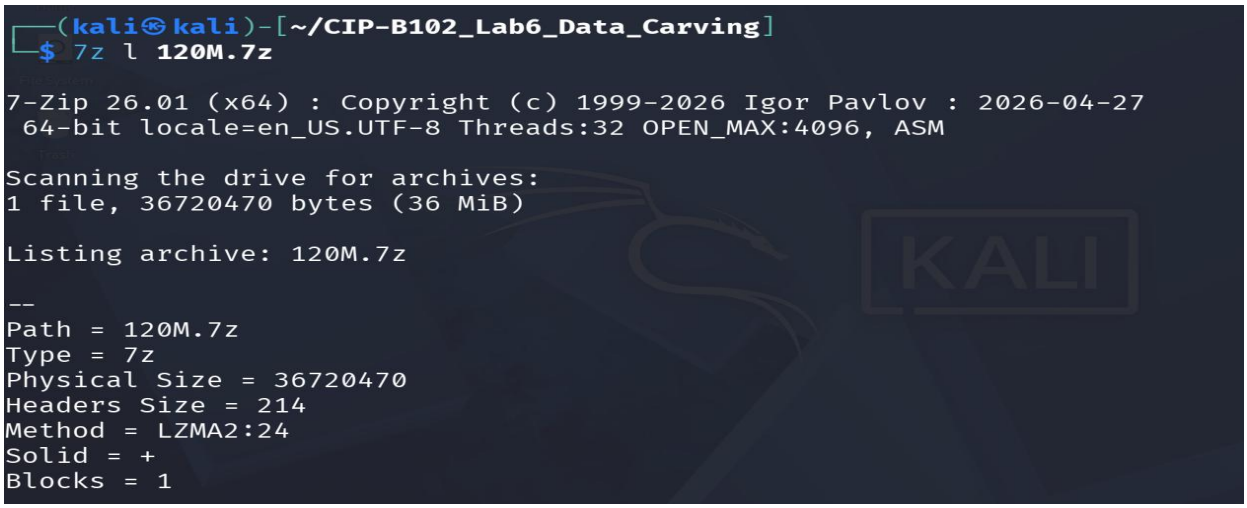

Figure 32. 7z l 120M.7z listing the archive header (7-Zip 26.01, LZMA2 method, solid archive, 1 block).

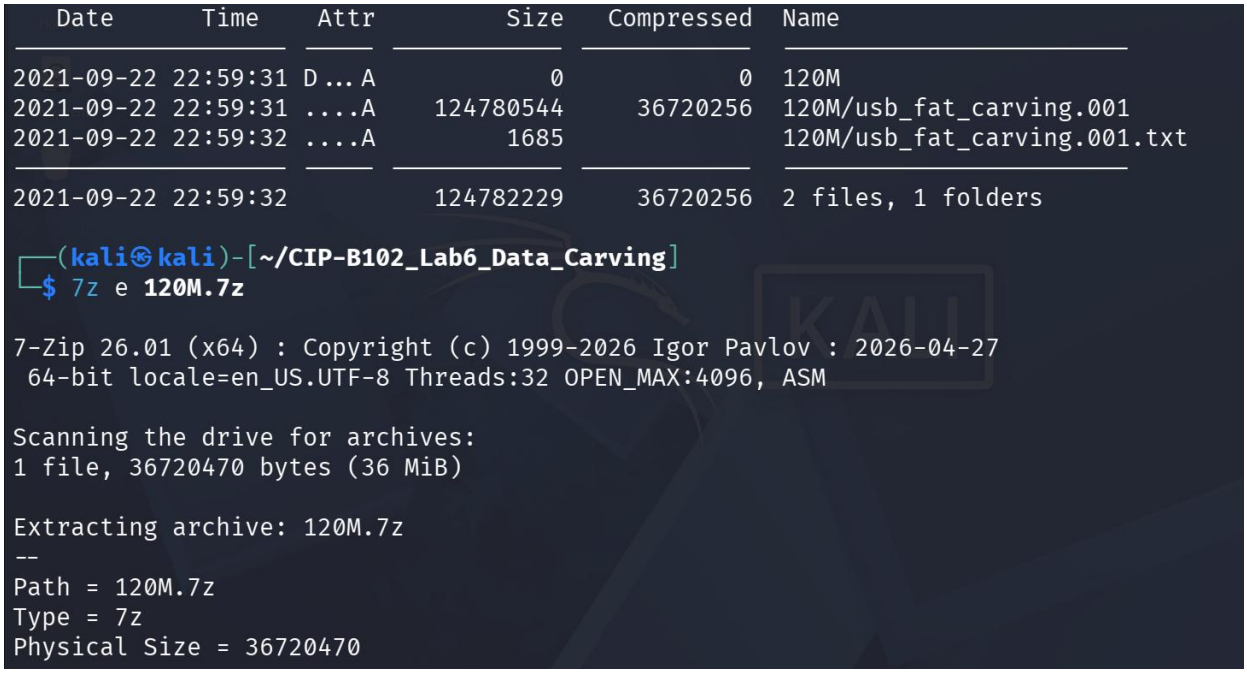

Figure 33. Archive contents confirmed: 120M/usb_fat_carving.001 (124,780,544 bytes) and 120M/usb_fat_carving.001.txt (1,685 bytes); extraction (7z e) started.

```
Headers Size = 214
Method = LZMA2:24
Solid = +
Blocks = 1

Everything is Ok

Folders: 1
Files: 2
Size:       124782229
Compressed: 36720470

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ ls -lh
total 166M
drwxrwxr-x 2 kali kali 4.0K Sep 22  2021 120M
-rw-rw-r-- 1 kali kali  36M Jul 12 06:41 120M.7z
drwxrwxr-x 2 kali kali 4.0K Jul 12 07:33 _120M.7z.extracted
-rw-rw-r-- 1 kali kali  24K Jul 12 07:07 Billing_Letter.doc
-rw-rw-r-- 1 kali kali 1.5M Jul 12 06:23 Ch01InChap01.dd
-rw-rw-r-- 1 kali kali  227 Jul 12 07:07 confirmation.txt
```

Figure 34. 7z e completed ('Everything is Ok', 2 files extracted) and ls -lh of the working directory showing the newly extracted files.

```
-rw-rw-r-- 1 kali kali 3.3M Jul 12 06:47 J_ub_law_hexdump_singleline.txt
-rw-rw-r-- 1 kali kali 3.3M Jul 12 06:46 J_ub_law_hexdump.txt
-rw-rw-r-- 1 kali kali 1.7M Jul 12 06:26 J_ub_law.jpg
-rw-rw-r-- 1 kali kali 1.7M Jul 12 06:48 J_ub_law_reverse_dump.jpg
-rw-rw-r-- 1 kali kali  121 Jul 12 07:08 letter1.txt
-rw-rw-r-- 1 kali kali    0 Jul 12 07:03 recovered_from_inode.bin
-rw-rw-r-- 1 kali kali 119M Sep 22  2021 usb_fat_carving.001
-rw-rw-r-- 1 kali kali 1.7K Sep 22  2021 usb_fat_carving.001.txt

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ file * | tee file_types_lab6.txt
120M:                          directory
120M.7z:                       7-zip archive data, version 0.4
_120M.7z.extracted:            directory
Billing_Letter.doc:            Composite Document File V2 Document, Little Endian, Os: Windows, V
ersion 4.10, Code page: 1252, Title: 13 October 2003, Author: Amelia Phillips, Template: Normal.dot
, Last Saved By: Amelia Phillips, Revision Number: 2, Name of Creating Application: Microsoft Word
9.0, Total Editing Time: 06:00, Create Time/Date: Sat Nov 23 03:06:00 2002, Last Saved Time/Date: F
ri Dec  9 14:50:00 2005, Number of Pages: 1, Number of Words: 118, Number of Characters: 678, Secur
ity: 0
Ch01InChap01.dd:               DOS/MBR boot sector, code offset 0x34+2, OEM-ID ",'k0nIHC" cached
```

Figure 35. Continued directory listing showing all working files, including the extracted usb_fat_carving.001 (119M).

```
by Windows 9M, root entries 224, sectors 2880 (volumes ⩽32 MB), sectors/FAT 9, sectors/track 18, d
os < 4.0 BootSector (0), FAT (12 bit by descriptor+sectors), followed by FAT
confirmation.txt:              ASCII text, with CRLF line terminators
J_ub_law_hexdump_singleline.txt: ASCII text, with very long lines (65536), with no line terminators
J_ub_law_hexdump.txt:          ASCII text
J_ub_law.jpg:                  JPEG image data, Exif standard: [TIFF image data, little-endian, d
irentries=16, width=4928, height=3280, bps=206, PhotometricInterpretation=RGB, manufacturer=NIKON C
ORPORATION, model=NIKON D4, orientation=upper-left], baseline, precision 8, 1920×1080, components 3
J_ub_law_reverse_dump.jpg:     JPEG image data, Exif standard: [TIFF image data, little-endian, d
irentries=16, width=4928, height=3280, bps=206, PhotometricInterpretation=RGB, manufacturer=NIKON C
ORPORATION, model=NIKON D4, orientation=upper-left], baseline, precision 8, 1920×1080, components 3
letter1.txt:                   ASCII text, with CRLF line terminators
recovered_from_inode.bin:      empty
usb_fat_carving.001:           DOS/MBR boot sector MS-MBR Windows 7 english at offset 0x163 "Inva
lid partition table" at offset 0x17b "Error loading operating system" at offset 0x19a "Missing oper
ating system", disk signature 0xa1159a00; partition 1 : ID=0xe, active, start-CHS (0x0,2,3), end-CH
S (0xe,254,63), startsector 128, 243584 sectors
usb_fat_carving.001.txt:       Unicode text, UTF-8 (with BOM) text, with CRLF line terminators
```

*Figure 36. file * output confirming file types for every working file, including usb_fat_carving.001 as a DOS/MBR boot sector with a Win95 FAT16 partition starting at sector 128.*

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ md5sum * | tee md5_after_extract_lab6.txt
md5sum: 120M: Is a directory
md5sum: _120M.7z.extracted: Is a directory
dfe7b5424e54cd1bf50d5df47aceeb3c  120M.7z
9fe241d0dde27e83442010b3eee5ad32  Billing_Letter.doc
a117773bcf1fc88ec0ab8e0a349fbbcb  Ch01InChap01.dd
18e391549e4a8bc990b264f590fb33bb  confirmation.txt
0653eae91c3bc8217f8b1d4bd39007e7  file_types_lab6.txt
7b99a4535ee37a9cfaa19a36b59b63a4  J_ub_law_hexdump_singleline.txt
a790fbdee8bcc5d30dd9c60119fcd7d6  J_ub_law_hexdump.txt
83a360ac7f7e0ca318e5bfe39f95f137  J_ub_law.jpg
83a360ac7f7e0ca318e5bfe39f95f137  J_ub_law_reverse_dump.jpg
49f68104660a091a4c015e86f3151237  letter1.txt
d41d8cd98f00b204e9800998ecf8427e  recovered_from_inode.bin
ba4a1d0ba49f4a6667b00a3b3e85e604  usb_fat_carving.001
568d6b183c03ff6f3e01e3915846be60  usb_fat_carving.001.txt
```

*Figure 37. md5sum * output for every file in the working directory, saved to md5_after_extract_lab6.txt for the evidence record.*

The extracted image filename matched the slide's usb_fat_carving.001, so it was used directly for the remaining TSK and scalpel steps.

```
IMAGE="usb_fat_carving.001" ls -lh "$IMAGE" img_stat "$IMAGE" mmls "$IMAGE" fls -o 128 "$IMAGE" fls -o 128 -d "$IMAGE"
```

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ IMAGE="usb_fat_carving.001"

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ ls -lh "$IMAGE"
-rw-rw-r-- 1 kali kali 119M Sep 22  2021 usb_fat_carving.001

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ img_stat "$IMAGE"
IMAGE FILE INFORMATION
--------------------------------------------
Image Type: raw

Size in bytes: 124780544
Sector size:    512
```

Figure 38. IMAGE variable set to usb_fat_carving.001; img_stat confirming a 124,780,544-byte raw image with 512-byte sectors.

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ mmls "$IMAGE"
DOS Partition Table
Offset Sector: 0
Units are in 512-byte sectors

     Slot    Start        End          Length       Description
000:  Meta    0000000000   0000000000   0000000001   Primary Table (#0)
001:  ------  0000000000   0000000127   0000000128   Unallocated
002:  000:000 0000000128   0000243711   0000243584   Win95 FAT16 (0x0e)

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ fls -o 128 "$IMAGE"
r/r 3:  USB         (Volume Label Entry)
d/d 6:  System Volume Information
r/r * 7:        _est
r/r 10: .dropbox.device
d/d * 13:       old_File_Carving_files
r/r * 15:       B_ub_poe4.bmp
r/r * 18:       B_zoom-eubie-mono.bmp
r/r * 21:       Ballardlab8.java
r/r * 24:       brittLab10.java
```

Figure 39. mmls output showing the DOS partition table: a Win95 FAT16 partition beginning at sector 128 (offset used for all subsequent fls commands), and the start of the fls -o 128 file listing.

```
r/r * 24:       brittLab10.java
r/r * 27:       DO_example.doc
r/r * 30:       DO_example2.doc
r/r * 33:       G_BuiltForThis.gif
r/r * 35:       G_zoom-sc.gif
r/r * 37:       H_Form.html
r/r * 39:       H_hello.html
r/r * 41:       J_ub_law.jpg
r/r * 44:       J_ub_night.jpg
r/r * 47:       nps-2008-jean_outlook.pst
r/r * 50:       P_CAS-zoom-6.png
r/r * 53:       P_MSB_1_zoom.png
r/r * 57:       pd_Evidence_search_techniques.pdf
r/r * 61:       pd_Forensic_Report_Template.pdf
r/r * 64:       pp_Number_Systems.pptx
r/r * 67:       pp_one_page.pptx
r/r 69: readme.docx
r/r 70: readme.txt
r/r * 71:       _tf_1.rtf
r/r * 74:       T_Eubie-iphone-5-8.tiff
r/r * 78:       T_youknowus-iphone-5-8.tiff
r/r * 81:       W_axloadcomplete.wav
```

*Figure 40. Continued fls -o 128 listing showing dozens of allocated and deleted (marked *) files on the USB volume, including images, documents, and archive files.*

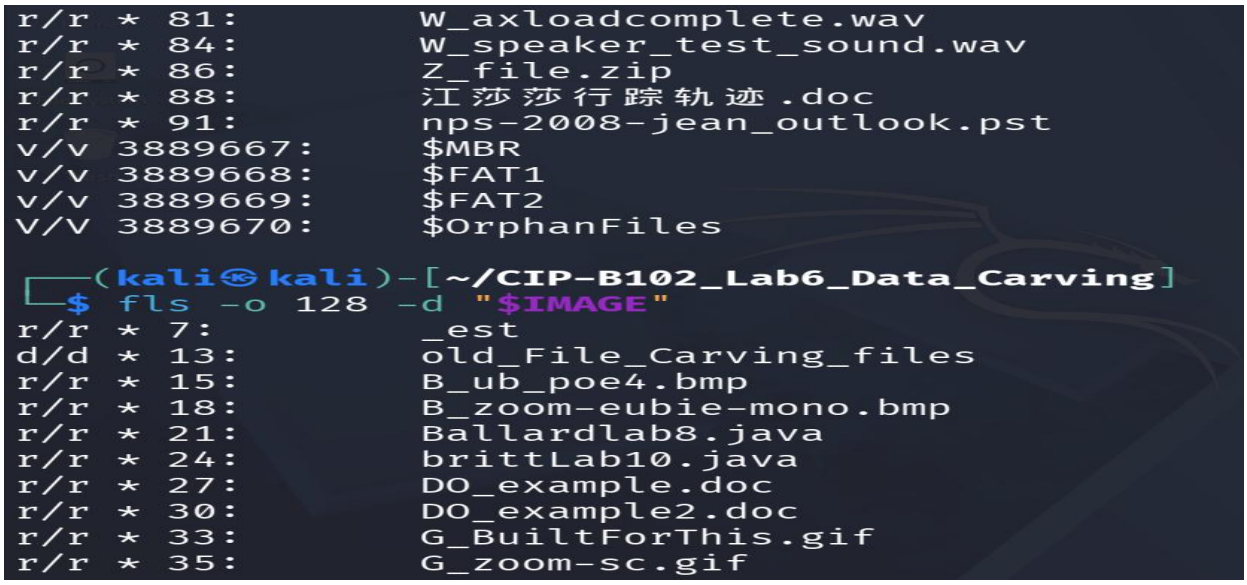

Figure 41. Further fls -o 128 output and the start of fls -o 128 -d, isolating only the deleted entries such as W_axloadcomplete.wav and Z_file.zip alongside FAT system entries ($MBR, $FAT1, $FAT2, $OrphanFiles).

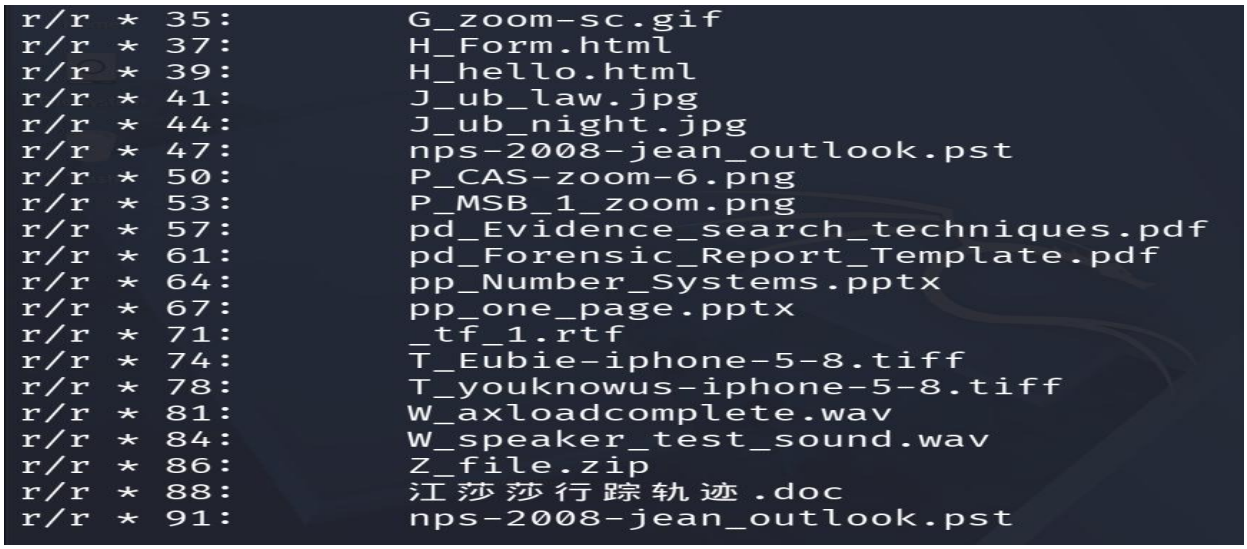

Figure 42. Continued fls -o 128 -d deleted-file listing for the USB volume.

mmls confirmed the USB image contains a single Win95 FAT16 partition starting at sector 128 with a length of 243,584 sectors; the offset of 128 used throughout this lab (and provided in the slide) matches this partition table exactly, so no adjustment was required.

4.8 Part G: Configuring Scalpel and Carving the USB Image

sudo cp /etc/scalpel/scalpel.conf /etc/scalpel/scalpel.conf.bak sudo nano /etc/scalpel/scalpel.conf

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ sudo cp /etc/scalpel/scalpel.conf /etc/scalpel/scalpel.conf.bak
[sudo] password for kali:

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ ls -lh /etc/scalpel/scalpel.conf*
-rw-r--r-- 1 root root 8.5K Sep 27  2025 /etc/scalpel/scalpel.conf
-rw-r--r-- 1 root root 8.5K Jul 12 08:25 /etc/scalpel/scalpel.conf.bak

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ sudo nano /etc/scalpel/scalpel.conf
```

Figure 43. scalpel.conf backed up before editing, and the configuration file opened in nano.

```
#----------------------------------------------------------------------
#
#
# AOL ART files
#       art     y       150000  \x4a\x47\x04\x0e        \xcf\xc7\xcb
#       art     y       150000  \x4a\x47\x03\x0e        \xd0\xcb\x00\x00
#
# GIF and JPG files (very common)
#       gif     y       5000000         \x47\x49\x46\x38\x37\x61        \x00\x3b
#       gif     y       5000000         \x47\x49\x46\x38\x39\x61        \x00\x3b
        jpg     y       5242880         \xff\xd8\xff???Exif             \xff\xd9    >
        jpg     y       5242880         \xff\xd8\xff???JFIF             \xff\xd9    >
#
#
# PNG

^G Help       ^O Write Out  ^F Where Is   ^K Cut       ^T Execute    ^C Location
^X Exit       ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line
```

Figure 44. Relevant lines enabled in scalpel.conf: the gif and jpg signature definitions, including the two jpg header/footer patterns (Exif-style and JFIF-style, both terminated by 0xFFD9) used for this lab.

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ sudo nano /etc/scalpel/scalpel.conf

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ IMAGE="usb_fat_carving.001"

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ ls -lh "$IMAGE"
-rw-rw-r-- 1 kali kali 119M Sep 22  2021 usb_fat_carving.001

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ rm -rf output_lab6_scalpel
```

Figure 45. Configuration saved; IMAGE variable re-confirmed; previous output_lab6_scalpel directory removed before the carving run.

```
scalpel "$IMAGE" -o output_lab6_scalpel
```

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ scalpel "$IMAGE" -o output_lab6_scalpel
Scalpel version 1.60
Written by Golden G. Richard III, based on Foremost 0.69.

Opening target "/home/kali/CIP-B102_Lab6_Data_Carving/usb_fat_carving.001"

Image file pass 1/2.
usb_fat_carving.001: 100.0% |******************************|  119.0 MB    00:00 ETA
Allocating work queues...
Work queues allocation complete. Building carve lists...
Carve lists built.  Workload:
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x45\x78\x69\x66" and footer "\xff\xd9" ⟶
 8 files
jpg with header "\xff\xd8\xff\x3f\x3f\x3f\x4a\x46\x49\x46" and footer "\xff\xd9" ⟶
 9 files
Carving files from image.
Image file pass 2/2.
usb_fat_carving.001: 100.0% |******************************|  119.0 MB    00:00 ETA
```

Figure 46. scalpel v1.60 starting the carve: it built two jpg work queues — 8 candidate files matching the Exif-style header/footer and 9 matching the JFIF-style header/footer.

```
Processing of image file complete. Cleaning up...
Done.
Scalpel is done, files carved = 17, elapsed = 0 seconds.

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ find output_lab6_scalpel -type f -maxdepth 3 -print
find: warning: you have specified the global option -maxdepth after the argument -ty
pe, but global options are not positional, i.e., -maxdepth affects tests specified b
efore it as well as those specified after it.  Please specify global options before
other arguments.
output_lab6_scalpel/audit.txt
output_lab6_scalpel/jpg-0-0/00000000.jpg
output_lab6_scalpel/jpg-0-0/00000002.jpg
output_lab6_scalpel/jpg-0-0/00000001.jpg
output_lab6_scalpel/jpg-0-0/00000006.jpg
output_lab6_scalpel/jpg-0-0/00000007.jpg
output_lab6_scalpel/jpg-0-0/00000004.jpg
output_lab6_scalpel/jpg-0-0/00000005.jpg
```

Figure 47. Scalpel completed ('files carved = 17'); find confirming the output structure, including audit.txt and the carved JPGs under output_lab6_scalpel/jpg-0-0/.

```
cat output_lab6_scalpel/audit.txt | head -80
```

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ cat output_lab6_scalpel/audit.txt | head -80

Scalpel version 1.60 audit file
Started at Sun Jul 12 08:31:40 2026
Command line:
scalpel usb_fat_carving.001 -o output_lab6_scalpel

Output directory: /home/kali/CIP-B102_Lab6_Data_Carving/output_lab6_scalpel
Configuration file: /etc/scalpel/scalpel.conf

Opening target "/home/kali/CIP-B102_Lab6_Data_Carving/usb_fat_carving.001"

The following files were carved:
File              Start                Chop            Length          Extracted Fr
om
00000010.jpg      3796992              NO          3148500             usb_fat_carv
ing.001
00000009.jpg       425822              NO          4875802             usb_fat_carv
```

Figure 48. audit.txt header (run timestamp, command line, configuration file used) and the start of the carved-file table (start offset, chop flag, length, source image).

```
00000007.jpg     21358592             NO          5183267             usb_fat_carv
ing.001
00000016.jpg     36671488             NO          2889262             usb_fat_carv
ing.001
00000015.jpg     36571136             NO          2989614             usb_fat_carv
ing.001
00000014.jpg     36393610             NO          3167140             usb_fat_carv
ing.001
00000013.jpg     36352448             NO          3208302             usb_fat_carv
ing.001
00000012.jpg     35590996             NO          3969754             usb_fat_carv
ing.001
00000011.jpg     35350242             NO          4210508             usb_fat_carv
ing.001


Completed at Sun Jul 12 08:31:40 2026
```

Figure 49. Remaining rows of the carved-file table in audit.txt, listing all 17 carved JPGs with their start offsets and extracted lengths.

```
find output_lab6_scalpel -type f -exec file {} \; find output_lab6_scalpel -type f -exec md5sum {} \; > carved_files_md5.txt find output_lab6_scalpel -type f \( -name "*.jpg" -o -name "*.JPG" \)
```

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ find output_lab6_scalpel -type f -exec file {} \;
output_lab6_scalpel/audit.txt: ASCII text
output_lab6_scalpel/jpg-0-0/00000000.jpg: JPEG image data, Exif standard: [TIFF imag
e data, big-endian, direntries=7, orientation=upper-left, xresolution=98, yresolutio
n=106, resolutionunit=2, software=Adobe Photoshop 21.1 (Macintosh), datetime=2020:04
:03 14:35:44], baseline, precision 8, 1920×1080, components 3
output_lab6_scalpel/jpg-0-0/00000002.jpg: JPEG image data, Exif standard: [TIFF imag
e data, big-endian, direntries=12, width=16550, height=9313, bps=0, PhotometricInter
pretation=RGB, orientation=upper-left]
output_lab6_scalpel/jpg-0-0/00000001.jpg: JPEG image data, Exif standard: [TIFF imag
e data, big-endian, direntries=7, orientation=upper-left, xresolution=98, yresolutio
n=106, resolutionunit=2, software=Adobe Photoshop 21.1 (Macintosh), datetime=2020:04
:03 11:54:50], baseline, precision 8, 1920×1080, components 3
output_lab6_scalpel/jpg-0-0/00000006.jpg: JPEG image data, Exif standard: [TIFF imag
e data, big-endian, direntries=7, orientation=upper-left, xresolution=98, yresolutio
n=106, resolutionunit=2, software=Adobe Photoshop 21.1 (Macintosh), datetime=2020:04
:23 15:24:33], baseline, precision 8, 1920×1080, components 3
output_lab6_scalpel/jpg-0-0/00000007.jpg: JPEG image data, Exif standard: [TIFF imag
```

Figure 50. file command confirming several carved outputs as valid JPEG images with Exif/TIFF metadata (e.g., Adobe Photoshop 21.1 software tags and embedded capture dates).

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ find output_lab6_scalpel -type f -exec md5sum {} \; > carved_files_md5.txt

┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ cat carved_files_md5.txt
e7164d558441306a3f2bd72f78776c31  output_lab6_scalpel/audit.txt
280d788eb3d1d48b93e88374440b1cc9  output_lab6_scalpel/jpg-0-0/00000000.jpg
4a5d89f1d59cc27959925a6a93fb8ed8  output_lab6_scalpel/jpg-0-0/00000002.jpg
e553e2ec5b6d2c67829a0edbcc7609cd  output_lab6_scalpel/jpg-0-0/00000001.jpg
23aebb41d1f892897e275e13145fead0  output_lab6_scalpel/jpg-0-0/00000006.jpg
80d6a778d127d09b03f01dc0821b0907  output_lab6_scalpel/jpg-0-0/00000007.jpg
85f5990b4cc8910d16c9a80b04f919a4  output_lab6_scalpel/jpg-0-0/00000004.jpg
8d6c3778abdb79fd36e07a79bd6abcc1  output_lab6_scalpel/jpg-0-0/00000005.jpg
d143fe524b792d62b55ad6b7e12c1dfa  output_lab6_scalpel/jpg-0-0/00000003.jpg
aec4abf7cb00ca5fcf54967d454a801d  output_lab6_scalpel/jpg-1-0/00000012.jpg
41cb821d0e0657ed0d2fabe09d685dfc  output_lab6_scalpel/jpg-1-0/00000010.jpg
6ea7a6c2e3988a8147e58caee15dc9d6  output_lab6_scalpel/jpg-1-0/00000008.jpg
```

Figure 51. md5sum computed for every carved file and saved to carved_files_md5.txt for the evidence archive.

```
┌──(kali㉿kali)-[~/CIP-B102_Lab6_Data_Carving]
└─$ find output_lab6_scalpel -type f \( -name "*.jpg" -o -name "*.JPG" \)
output_lab6_scalpel/jpg-0-0/00000000.jpg
output_lab6_scalpel/jpg-0-0/00000002.jpg
output_lab6_scalpel/jpg-0-0/00000001.jpg
output_lab6_scalpel/jpg-0-0/00000006.jpg
output_lab6_scalpel/jpg-0-0/00000007.jpg
output_lab6_scalpel/jpg-0-0/00000004.jpg
output_lab6_scalpel/jpg-0-0/00000005.jpg
output_lab6_scalpel/jpg-0-0/00000003.jpg
output_lab6_scalpel/jpg-1-0/00000012.jpg
output_lab6_scalpel/jpg-1-0/00000010.jpg
output_lab6_scalpel/jpg-1-0/00000008.jpg
output_lab6_scalpel/jpg-1-0/00000014.jpg
output_lab6_scalpel/jpg-1-0/00000016.jpg
output_lab6_scalpel/jpg-1-0/00000009.jpg
output_lab6_scalpel/jpg-1-0/00000015.jpg
```

Figure 52. Final find listing of all 17 carved .jpg files across the jpg-0-0 and jpg-1-0 output subfolders.

Visual verification: 00000000.jpg and 00000001.jpg were both confirmed by file as valid, well-formed JPEG images (baseline, 1920x1080, 3 components) and opened correctly in an image viewer. 00000002.jpg was also identified as JPEG image data by file, but reported implausible dimensions (width 16550 x height 9313) a sign of a partially carved or corrupted file, most likely caused by scalpel reaching its configured maximum carve length before the true EOI footer, or by two adjacent JPEGs being carved as one. This is noted under Limitations below.

5. Findings

Finding Item	Student Response
Original evidence file names	Ch01InChap01.dd, J_ub_law.jpg, 120M.7z (120M.7z extracted to usb_fat_carving.001)
Original evidence hash values	See Section 3.1 hash table (MD5/SHA-256 for all three downloaded files)
Partition offset used	128 sectors (Win95 FAT16 partition confirmed by mmls on usb_fat_carving.001)
Scalpel configuration file types enabled	jpg (two header/footer patterns: Exif-style FFD8FF3F3F3F4578696600...FFD9 and JFIF-style FFD8FF3F3F3F4A46494600...FFD9)
Number of carved files recovered	17 files (scalpel reported: files carved = 17)
Number of carved JPG files that opened successfully	At least 2 confirmed valid and viewable (00000000.jpg, 00000001.jpg); one additional file (00000002.jpg) carved with implausible dimensions, indicating an incomplete/corrupted carve
Limitations observed	Carved files lost their original names, paths, and timestamps; File_carving.docx was unavailable so the manual dd-offset extraction step was substituted; binwalk reported false-positive signatures inside the 7z LZMA2 stream

6. Validation

Every recovered or reconstructed file in this lab was validated using at least one of: the file command (to confirm the recovered content matches the expected file type/signature), md5sum/sha256sum (to confirm byte-for-byte integrity where an original comparison file existed), and direct visual inspection (opening recovered JPGs in an image viewer).

- The reverse-dumped JPEG (J_ub_law_reverse_dump.jpg) produced an MD5/SHA-256 hash identical to the original J_ub_law.jpg, confirming a lossless hex-to-binary round trip.

- icat-recovered files (Billing_Letter.doc, confirmation.txt, letter1.txt) were each confirmed by file as valid documents/text of the expected type, and their content was readable with cat/a text or document viewer.
- Scalpel-carved JPGs were validated with file, which confirmed valid JPEG/Exif structure for most carved files; two files were opened successfully in an image viewer, while one showed signs of an incomplete carve.
- All working-directory files were re-hashed after extraction (md5_after_extract_lab6.txt) to preserve a complete chain-of-custody record for this lab.

7. Limitations

- Data carving with scalpel recovers file content only; it cannot recover original filenames, folder paths, or file-system timestamps, because that information lives in file-system metadata, not in the byte pattern being carved. All 17 carved files therefore received generic sequential names (e.g., 00000000.jpg).
- File_carving.docx, used in the slide's manual dd-offset extraction demonstration, had no download URL in the slide material and was not available in this environment, so that exact manual step could not be reproduced verbatim; binwalk/dd practice was completed using J_ub_law.jpg and the 120M.7z/usb_fat_carving.001 evidence instead, as permitted by the lab instructions.
- binwalk reported several Zip-archive and JBOOT-style signature matches inside 120M.7z that do not correspond to real embedded files; these are heuristic false positives arising from byte patterns inside the compressed LZMA2 stream, and automatic extraction (binwalk -e) only produced a handful of small unrelated fragments, not the real usb_fat_carving.001 content.
- One scalpel-carved file (00000002.jpg) reported implausible image dimensions, suggesting the carve either ran past the true EOI marker (picking up a length limit or an adjacent JPEG) or captured a truncated/corrupted stream; this is a known limitation of signature-based carving when files are fragmented or stored non-contiguously on disk.
- Only jpg (and optionally gif) signatures were enabled in scalpel.conf for this lab; any other embedded file types present on the USB image outside the enabled signatures would not have been carved.

8. Conclusion

This lab demonstrated the full spectrum of file recovery techniques used in digital forensics, from simple hex-level signature inspection with xxd, through metadata-assisted recovery of deleted-but-still-referenced files with The Sleuth Kit, to true signature-based carving of files with no remaining metadata using scalpel, plus embedded-content discovery with binwalk. The practical difference between these techniques was made clear: TSK's icat/istat could still recover original filenames and timestamps for the deleted entries on Ch01InChap01.dd because their directory entries were merely unallocated, not overwritten, whereas scalpel's byte-pattern carving of the USB image recovered image content with no names or timestamps at all, because it worked purely from JPEG header/footer signatures with no file-system context. Hash verification (md5sum/sha256sum) at each stage confirmed that reconstructed and recovered files were bit-for-bit accurate where a comparison baseline existed. Overall, the lab reinforced why a forensic analyst must document every command, hash value, and configuration choice: carving is a powerful last resort for recovering content, but it cannot substitute for intact metadata when filenames, paths, or timestamps are evidentially important.

9. Reflection (Word Limit: 250 Words)

Data carving cannot always recover filenames and metadata because it works from a fundamentally different layer of the file system than normal recovery. A file system like FAT stores a file's name, size, and timestamps in a separate directory entry, and it stores the file's content in a chain of clusters linked through the File Allocation Table. When a file is deleted, the operating system typically frees the directory entry and FAT chain but leaves the cluster data untouched until it is overwritten. Tools like TSK's icat and istat can still recover names and timestamps in this situation, as seen in this lab with Billing_Letter.doc and confirmation.txt, because the directory entry itself, though marked unallocated, was still present and intact.

Carving tools such as scalpel work differently: they scan raw, unstructured bytes for recognizable header and footer signatures (such as JPEG's FFD8 and FFD9) with no reference to any file system at all. This is what makes carving so powerful, since it can recover data from formatted drives, corrupted file systems, or unallocated space where no directory entries survive; but it is also why carving can never reconstruct a name, path, or timestamp, since none of that information is encoded in the file's raw content. This lab illustrated the contrast directly: all 17 JPEGs carved from the USB image by scalpel received generic numbered names with no original metadata, while files recovered by TSK from the still-intact FAT

directory entries kept their real names and timestamps. Analysts must therefore treat carved evidence as content-only and corroborate identity, ownership, or timing through other means whenever possible.

10. Deliverables Checklist

- PDF/Word lab report (this document) CIP-B102_Lab6_[Firstname]_[Lastname]
- ZIP archive containing recovered/carved files: Billing_Letter.doc, confirmation.txt, letter1.txt, J_ub_law_reverse_dump.jpg, and the output_lab6_scalpel/ folder (17 carved JPGs + audit.txt)
- Hash text files: md5_after_extract_lab6.txt, carved_files_md5.txt, file_types_lab6.txt
- Command log/appendix all commands used are included inline throughout Section 4 above
- This reflection (Section 9, ≤250 words)